

第8章 网络通信

socket 迁移 · TCP/UDP · 粘包与协议设计 · requests · 最小 Web 服务

本章目标:以 C 的 socket API 为对照,讲透 Python 网络编程:TCP/UDP 客户端与服务器、粘包问题与长度前缀协议、超时与异常处理,再上层到 requests 调用 HTTP API、http.server 搭建最小 Web 服务。学完你能写出可靠、有超时、有重试的生产级网络程序。

直觉起点:C 里网络是"文件描述符 + 一串套接字函数";Python 的 socket 模块 API 与 C 几乎一一对应,迁移成本极低。真正的难点不在 API,而在网络本身的三条铁律:数据可能不来、可能迟到、可能不完整。所有生产实践——超时、重试、长度前缀——都是对这三条铁律的回应。

前置要求:第7章文件与编码(网络收发的本质是字节流)、第5章 with 与资源管理。

本章路线:第一阶段(1.5 小时)读 §8.1–§8.3,理解 socket 流程与 TCP 语义;第二阶段(1.5 小时)读 §8.4–§8.5,掌握协议设计与 UDP;第三阶段(2 小时)读 §8.6–§8.8,完成 HTTP 层并做章末实验。

- 本章知识导图
- └─ 8.1 socket 迁移 ── C 到 Python 的一一对应
 - └─ 8.2 TCP 客户端 ── connect、收发与超时
 - └─ 8.3 TCP 服务器 ── bind、listen、accept 循环
 - └─ 8.4 粘包与协议 ── 长度前缀解决一切
 - └─ 8.5 UDP ── 无连接的取舍
 - └─ 8.6 HTTP 客户端 ── urllib 与 requests
 - └─ 8.7 http.server ── 40 行内的 Web 服务
 - └─ 8.8 生产实践 ── 超时、重试与异常

§8.1 socket 迁移:API 几乎一一对应

C 的 socket 编程流程:创建套接字、connect/bind、send/recv、close。Python 的 **socket** 模块就是这套 API 的直译,连常量名都一样。先看两端完整的 TCP 客户端对照:

C(socket + connect)	Python(socket)
socket(AF_INET, SOCK_STREAM, 0)	socket.socket(socket.AF_INET, socket.SOCK_STREAM)
struct sockaddr_in addr; ...	sock.connect(("host", 8080)) ──元组即地址
send(buf, len, 0) 返回字节数	sock.sendall(data) ──一次发完,内部循环
recv(buf, N, 0) 返回字节数	sock.recv(1024) 返回 bytes,空串=对端关闭
close(fd)	sock.close(),配 with 自动关闭
errno 判断错误类型	异常体系:ConnectionRefusedError 等

Python 的三个语法级改进值得记住:地址用元组 ("host", port),不用手填 sockaddr;发送用 **sendall** 而不是 send(后者可能只发一半);套接字支持 with,退出自动关闭。其余流程——connect、bind、listen、accept——与 C 完全同构。

迁移时的心理建设:Python 屏蔽了 C 里最磨人的部分——地址结构体、字节序转换、errno 错误分支——但网络语义(流、阻塞、半关闭)完全一致。你在 C 里学会的每一个网络概念,在 Python 里都能原样复用,差别只在代码量少了一个量级。

§8.2 TCP 客户端:连接、收发与超时

```
import socket

with socket.create_connection(("127.0.0.1", 9000), timeout=5) as s:
    # create_connection 自动完成 getaddrinfo + connect
    s.sendall(b"GET /time HTTP/1.0\r\nHost: localhost\r\n\r\n")

    chunks = []
    while True:
        data = s.recv(4096)
        if not data:           # 空 bytes:对端已关闭连接
            break
        chunks.append(data)
    print(b"".join(chunks).decode("utf-8", errors="replace"))
```

误区:recv 一次就想收完。 TCP 是字节流,recv 返回多少完全由网络与内核决定——可能只有 1 字节,也可能分批到达。recv(4096) 只保证"最多 4096",不保证"你要的都在里面"。收消息的唯一正确姿势是循环 recv,直到满足协议条件(定长、长度前缀或连接关闭)。

create_connection 是生产首选:它内部处理了 IPv4/IPv6 解析与自动重试,还接受 timeout 参数。超时是网络程序的第一安全阀:不设超时,服务端挂死时客户端会无限期阻塞,线程、连接全部耗尽。超时异常是 socket.timeout,记得捕获并转成业务错误。

超时单位是秒,可以是浮点数。设了超时后,阻塞的 recv/accept 到点抛 socket.timeout,连接并未失效——但生产上超时后重试更稳妥。调试网络程序时先把超时调大避免误判,再谈协议正确性;先确认连接通不通,再排查协议细节。

§8.3 TCP 服务器:bind、listen、accept

```
import socket

def handle(conn: socket.socket, addr: tuple) -> None:
    """处理单个客户端连接:回显收到的内容"""
    with conn:
        print(f"连接来自 {addr}")
        while True:
            data = conn.recv(4096)
            if not data:
                break
            conn.sendall(data)      # 原样回显

def main() -> None:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 9000))
        srv.listen(8)              # 等待队列长度
        print("监听 9000 端口...")
        while True:
            conn, addr = srv.accept() # 阻塞等待新连接
            handle(conn, addr)        # 串行处理(演示用)

if __name__ == "__main__":
    main()
```

误区:accept 的连接忘了 close。 accept 返回的新连接是独立资源,用完不关会堆积,达到上限后新连接全部失败。用 with conn: 包裹处理逻辑即可自动关闭。另一个经典坑:服务器进程重启时 bind 报 Address already in use——setsockopt(SO_REUSEADDR, 1) 是服务器标配,否则调试期频繁被杀重启。

上面的服务器是串行的:一个客户端占着连接,其他客户端全部排队。生产要同时服务多个客户端,有两条路:每连接开线程(第9章讲),或事件循环(asyncio)。现在先理解串行版,它把 accept 循环讲清楚了。

理解 accept 的语义:它在监听队列上取一个已完成握手的连接;队列满时内核会丢弃新的握手请求。listen(n) 的 n 是等待队列深度,不是并发数。生产并发要靠线程或事件循环——第9章会把 threading 与 asyncio 讲透,届时回头看这个串行版,架构演进脉络就清楚了。

§8.4 粘包与协议设计:长度前缀

初学 TCP 最常见的困惑是"粘包":两次 send,对端一次 recv 全收到,以为消息黏在一起了。真相是 TCP 不保消息边界——它只保证字节有序到达。send("AB") 与 send("CD") 可能合并传输,也可能拆成三次。解决方案不是"避免粘包"(做不到),而是**设计协议,让接收方能确定消息边界**。

```
import struct

# 协议:4 字节长度前缀(大端) + 消息体
def send_msg(sock: socket.socket, payload: bytes) -> None:
    header = struct.pack(">I", len(payload)) # 定长 4 字节
    sock.sendall(header + payload)

def recv_exact(sock: socket.socket, n: int) -> bytes:
    """收满 n 字节为止—recv 可能分批到达"""
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("对端提前关闭")
        buf.extend(chunk)
    return bytes(buf)

def recv_msg(sock: socket.socket) -> bytes:
    header = recv_exact(sock, 4)
    (length,) = struct.unpack(">I", header)
    return recv_exact(sock, length)
```

C(手工组包)	Python(长度前缀)
htonl 转网络字节序,memcpy 进缓冲	struct.pack(">I", n) 一步完成
recv 循环自己写,容易漏边界	recv_exact 收满 n 字节,复用一次
缓冲区管理易越界	bytearray 自动扩容,长度即边界
每种协议重写一遍边界逻辑	send_msg/recv_msg 一对函数通用所有消息

长度前缀是二进制协议的基础设施,HTTP 的 Content-Length、WebSocket 的帧头、主流 RPC 框架都基于同一思想。上限检查别忘了:recv_exact 前先校验 length 不超约定上限(如 10 MB),否则恶意端一个 4 GB 的长度字段就能拖垮接收方内存。

协议设计三原则:长度字段定宽(4 字节足以表达 4 GB)、先验后收(长度超限直接拒绝)、预留版本字段(将来加字段不破坏旧客户端)。内部系统协议别贪多,够用即止——每多一个字段,就要多写一组兼容测试。

§8.5 UDP:无连接的取舍

UDP 无连接、不保证到达、不保证顺序,换来低延迟与无握手开销。Python 的用法与 TCP 对称但更简单——没有 accept,一个套接字收发都行:

```
import socket

# 服务器:绑定端口,循环收包
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    while True:
        data, addr = srv.recvfrom(4096)      # 一次收一个完整数据报
        print(f"{addr} 发来 {data}")
        srv.sendto(data, addr)              # 回给同一个地址

# 客户端:无需连接,直接发
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cli:
    cli.sendto(b"ping", ("127.0.0.1", 9001))
    reply, _ = cli.recvfrom(4096)
```

注意 `recvfrom` 返回 (数据, 对端地址), 一个数据报对应一次 `recvfrom`——UDP 天然保留消息边界, 没有粘包问题。代价是丢包与乱序: 生产 UDP 程序要自带序号、重传与超时确认(或者直接用 QUIC 这类成熟协议, 别自己造轮子)。

什么场景选 UDP: 实时音视频、游戏位置同步、DNS、日志采集——丢一帧无所谓, 延迟敏感。什么场景绝不选 UDP: 交易、订单、配置下发——凡是“丢一条消息会出事故”的业务, 老老实实用 TCP 或可靠传输协议, 这是网络选型的第一课。

§8.6 HTTP 客户端: `urllib` 与 `requests`

调 HTTP API 是网络编程的最高频场景。标准库 `urllib` 能干活但笨重; 生产事实标准是第三方库 `requests` (`pip install requests`)。它的三个核心能力: `get/post`、JSON 自动编解码、响应状态检查:

```
import requests

resp = requests.get(
    "https://api.example.com/v1/orders/A1001",
    timeout=(3.05, 10),          # (连接超时, 读取超时)
)
resp.raise_for_status()         # 4xx/5xx 抛 HTTPError

order = resp.json()             # 自动按 Content-Type 解析
print(order["status"])

resp2 = requests.post(
    "https://api.example.com/v1/orders",
    json={"sku": "P-001", "qty": 2}, # json= 自动序列化并带头
    timeout=10,
)
print(resp2.status_code, resp2.json())
```

误区: 请求不带 `timeout`。 不设 `timeout` 的 `requests` 请求可能永远挂着(默认无超时!), 线程池、任务队列、重试循环全部被一个慢服务拖死。生产规范: 连接超时 3 秒、读取超时按业务给 10–30 秒, 并捕获 `requests.Timeout` 单独处理——超时不该被当作普通网络错误。

`requests` 的其他生产要点: `params=` 传查询参数(自动 URL 编码)、`headers=` 传认证头、`verify=False` 仅测试环境用并配警告抑制、`Session` 复用连接池(大量请求时性能差距明显)。

requests 的响应对象是“一次性的”:resp.content 与 resp.text 读写的是同一块解码缓冲,大响应别反复取。流式大文件下载用 iter_content(chunk_size) 按块迭代,避免整包进内存——这是网络版的 for line in f,与第7章的大文件哲学一脉相承。

§8.7 http.server:40 行内的最小 Web 服务

标准库 http.server 可以在一分钟内部署一个能处理业务逻辑的 HTTP 服务——适合内部工具、测试桩、原型。核心是继承 BaseHTTPRequestHandler 覆写 do_GET/do_POST:

```
import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

ORDERS = {}                                # 内存存储,演示用

class Handler(BaseHTTPRequestHandler):
    def do_GET(self) -> None:
        if self.path == "/health":
            body = json.dumps({"status": "ok"}).encode("utf-8")
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
        else:
            self.send_error(404, "未找到资源")

    def do_POST(self) -> None:
        if self.path == "/orders":
            length = int(self.headers.get("Content-Length", 0))
            data = json.loads(self.rfile.read(length))
            oid = f"B{len(ORDERS) + 1:04d}"
            ORDERS[oid] = data
            body = json.dumps({"order_id": oid}).encode("utf-8")
            self.send_response(201)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
        else:
            self.send_error(404)

if __name__ == "__main__":
    ThreadingHTTPServer(("127.0.0.1", 8080), Handler).serve_forever()
```

C(手写 HTTP 服务器)	Python(http.server)
解析请求行、头部、Content-Length	BaseHTTPRequestHandler 已解析,self.path/self.headers 就绪
手动拼响应头与状态行	send_response/send_header/end_headers 三连
线程模型自己管	ThreadingHTTPServer 每连接一线程
读请求体要自写循环	<code>self.rfile.read(length)</code> 按长度读取

BaseHTTPRequestHandler 的 self.path 是请求行里的原始路径,self.headers 是邮件头格式的头部容器;query string 要自己从 self.path 中解析。模板渲染、静态文件、表单解析这些能力标准库给不了——原型验证阶段手写够用,规模上来就换框架。

误区:把 http.server 当生产 Web 服务器。 它只监听一个端口、无 HTTPS、无并发调优,只适合内网工具与测试。生产服务请用 FastAPI/Flask 加成熟服务器,或直接交给云平台。认清边界,工具才不伤人。

§8.8 生产实践:超时、重试与异常

网络代码的三条铁律(可能不来、可能迟到、可能不完整)在生产中落到三条纪律:

```
import time
import requests
from requests.exceptions import RequestException

MAX_RETRIES = 3
BACKOFF = 1.0

def fetch_with_retry(url: str, timeout: float = 10) -> dict:
    """带超时与退避重试的 GET;指数退避避免雪崩"""
    for attempt in range(MAX_RETRIES):
        try:
            resp = requests.get(url, timeout=timeout)
            resp.raise_for_status()
            return resp.json()
        except (requests.Timeout, requests.ConnectionError):
            if attempt == MAX_RETRIES - 1:
                raise RuntimeError(f"请求 {url} 连续失败") from None
            time.sleep(BACKOFF * (2 ** attempt)) # 1s, 2s
        except requests.HTTPError as e:
            if resp.status_code in (429, 500, 502, 503, 504):
                time.sleep(BACKOFF * (2 ** attempt))
                continue
            raise RuntimeError(f"HTTP {resp.status_code}: {url}") from e
    raise RuntimeError("不可达") # 不会执行到,类型安全兜底
```

工程建议 1:重试只对“可重试”的错误生效:超时、连接拒绝、5xx、429 可以重试;4xx 业务错误重试也没用,直接失败并记日志。重试要有上限与退避,否则下游故障时你的重试风暴会把它彻底打垮——这是生产事故的经典剧本。

工程建议 2:网络异常永远带上下文:捕获后重新抛出时注明 URL、方法、耗时,如 `raise RuntimeError(f"GET {url} 超时") from e`。from 保留原始异常链,排障时一眼看到根因;绝不裸 except 吞掉异常——静默失败是分布式系统最贵的 bug。

总结网络编程的黄金三角:超时兜底(永不无限等待)、协议显式(边界、长度、版本)、异常可查(上下文、异常链、日志)。把这三条写进团队代码规范,网络事故率会直线下降——它们本质上是把"网络不可靠"这个事实内化成代码结构。

本章速查表

场景	写法
TCP 客户端	<code>socket.create_connection((host, port), timeout=5)</code>
发送全部字节	<code>sock.sendall(data)</code> ,不用 <code>send</code>
循环接收	<code>while True: d = sock.recv(4096); if not d: break</code>
TCP 服务器	<code>bind</code> → <code>listen</code> → <code>accept</code> ,with 管理连接
端口复用	<code>srv.setsockopt(SO_REUSEADDR, 1)</code>
定长收发	4 字节大端长度前缀 + <code>recv_exact</code>
UDP 收	<code>data, addr = srv.recvfrom(4096)</code>
UDP 发	<code>sock.sendto(data, addr)</code>
HTTP GET	<code>requests.get(url, timeout=(3, 10))</code>
HTTP POST	<code>requests.post(url, json={...}, timeout=10)</code>
状态检查	<code>resp.raise_for_status()</code>
最小 Web 服务	<code>BaseHTTPRequestHandler</code> + <code>ThreadingHTTPServer</code>
读请求体	<code>self.rfile.read(int(self.headers["Content-Length"]))</code>
超时处理	捕获 <code>requests.Timeout</code> ,单独分支
重试	指数退避,只重试超时/5xx/429
异常链	<code>raise RuntimeError("...") from e</code>

最小必会清单

1. 能默写 `create_connection` + `sendall` + 循环 `recv` 的 TCP 客户端骨架
2. 能写出 `bind/listen/accept` 的服务器骨架,并说出两个经典坑(忘 `close`、端口占用)
3. 能解释"粘包"的本质,并用长度前缀协议实现 `send_msg/recv_msg`
4. 能说出 UDP 与 TCP 的取舍,以及 `recvfrom/sendto` 的用法
5. 能用 `requests` 完成带超时、状态检查、JSON 解析的 API 调用
6. 能用 `http.server` 在 40 行内搭出 GET/POST 的最小服务

自测题

Q

Q1. 为什么 `sendall` 比 `send` 可靠? `recv` 一次返回 100 字节,能说明对端只发了 100 字节吗?

答案:send 只尝试发送,返回实际发送字节数,可能小于请求长度(发送缓冲满或中断);sendall 内部循环发送直到全部完成或出错。不能——TCP 是字节流,recv 一次返回值只代表"当前内核缓冲区可用数据",可能与发送方的单次 send 无关,必须按协议边界循环接收。

Q

Q2. 服务器运行中端口报 "Address already in use",可能原因与解法?

答案:上一进程的套接字处于 TIME_WAIT 状态,或另一进程占用该端口。解法:① 服务器创建时 setsockopt(SO_REUSEADDR, 1) 允许重用;② 确认旧进程已退出(或端口被其他程序占用,用 netstat/ss 排查)。SO_REUSEADDR 是服务器标配,客户端不需要。

Q

Q3. 长度前缀协议中,收方为什么要先 recv_exact(4) 再 recv_exact(length)?直接把两段合并 recv 不行吗?

答案:不行,因为 recv 不保证一次拿到任意指定长度。先收满固定的 4 字节头才能解出消息长度,再按该长度收满消息体;两个阶段都依赖"收满为止"的循环(recv_exact)。合并 recv 会出现头只到一半、体只到一半、头体一起来等各种情况,只有循环收满才能统一处理。

Q

Q4. requests.get 不设 timeout 会怎样?连接超时与读取超时分别指什么?

答案:默认无超时,请求可能无限期挂起,线程与连接被耗尽。连接超时指建立 TCP 连接的最长等待(如 3 秒);读取超时指连接建立后两次数据到达之间的最长间隔(如 10 秒)。requests 用元组 (connect, read) 分别设置,生产必须显式给出。

Q

Q5. 一次请求返回 500 后,直接原样重试 3 次,有什么风险?正确做法?

答案:风险:若服务端正在过载,你的重试会加大压力,且无退避的瞬时重试几乎必然再失败,形成重试风暴。正确做法:只对可重试错误(超时、连接错误、5xx、429)重试,指数退避(1s、2s、4s)并加随机抖动,设上限次数;4xx 业务错误不重试,立即失败并记录。

Q

Q6. BaseHTTPRequestHandler 里,为什么读请求体要用 Content-Length 而不是 recv 到空?

答案:HTTP 请求体是定长传输(Content-Length 声明)或分块传输;请求结束后连接可能复用,recv 到空要等对端关闭连接,而 keep-alive 下对端不会关闭——直接死等。正确姿势是读满 Content-Length 声明的字节数(与 recv_exact 同理),这也是 HTTP 语义下"长度前缀"的体现。

章末实验题:带协议的文件传输 + 统计 API

实验 8.1 长度前缀文件传输与 HTTP 统计接口(覆盖:\$8.2-\$8.7)

需求:① 服务器端:监听 9100,用长度前缀协议接收"文件名(短串)+文件内容",存到 receive/ 目录;② 客户端:把本地任意文件按协议发过去,打印传输字节数与耗时;③ 服务器端另开 8081 端口提供 HTTP API:GET /files 返回已收文件列表

JSON,GET /files/<名> 返回文件内容;④ 双方都设置超时与异常处理,服务器打印每笔传输的日志;⑤ 用 requests 写一个测试脚本调用 API 验证。

覆盖知识点:TCP 客户端/服务器(\$8.2、\$8.3)/长度前缀协议与 recv_exact(\$8.4)/http.server 最小服务(\$8.7)/超时与异常链(\$8.8)。

实验环境:Python 3.10+,两个终端分别运行服务端与客户端。

```

# file_server.py — 长度前缀文件接收 + HTTP 统计 API
import json
import socket
import struct

from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path

HOST, PORT = "127.0.0.1", 9100
SAVE_DIR = Path("receive")
SAVE_DIR.mkdir(exist_ok=True)
MAX_LEN = 10 * 1024 * 1024          # 协议上限 10 MB


def recv_exact(conn: socket.socket, n: int) -> bytes:
    buf = bytearray()
    while len(buf) < n:
        chunk = conn.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("对端提前关闭")
        buf.extend(chunk)
    return bytes(buf)


def recv_str(conn: socket.socket) -> str:
    (length,) = struct.unpack(">I", recv_exact(conn, 4))
    if length > 256 or length < 1:
        raise ValueError(f"非法文件名长度: {length}")
    return recv_exact(conn, length).decode("utf-8")


def serve_tcp() -> None:
    """协议: [4字节文件名长][文件名][4字节内容长][内容]"""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind((HOST, PORT))
        srv.listen(4)
        print(f"文件服务监听 {HOST}:{PORT}")
        while True:
            conn, addr = srv.accept()
            with conn:
                conn.settimeout(30)
                try:
                    name = recv_str(conn)
                    (size,) = struct.unpack(">I", recv_exact(conn, 4))
                    if size > MAX_LEN:
                        raise ValueError(f"文件过大: {size}")
                    content = recv_exact(conn, size)
                    (SAVE_DIR / name).write_bytes(content)
                    print(f"[{addr}] 收到 {name} ({size} 字节)")
                except (ConnectionError, ValueError, socket.timeout) as e:
                    print(f"[{addr}] 传输失败: {e}")

```

```

class FileAPI(BaseHTTPRequestHandler):
    """GET /files 列表;GET /files/<name> 取文件"""

    def _json(self, code: int, obj: dict) -> None:
        body = json.dumps(obj).encode("utf-8")
        self.send_response(code)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self) -> None:
        if self.path == "/files":
            names = [p.name for p in SAVE_DIR.iterdir()]
            self._json(200, {"files": names})
            return
        if self.path.startswith("/files/"):
            name = self.path.removeprefix("/files/")
            target = SAVE_DIR / name
            if target.is_file():
                self._json(200, {"name": name,
                                "size": target.stat().st_size})
            else:
                self._json(404, {"error": "文件不存在"})
            return
        self._json(404, {"error": "未找到"})

    def log_message(self, fmt: str, *args) -> None:
        print(f"[http] {self.address_string()} {fmt % args}")

def main() -> None:
    import threading
    tcp = threading.Thread(target=serve_tcp, daemon=True)
    tcp.start()
    httpd = ThreadingHTTPServer(("127.0.0.1", 8081), FileAPI)
    print("HTTP API 监听 8081")
    httpd.serve_forever()

if __name__ == "__main__":
    main()

```

客户端与验证:客户端用 `create_connection` 连接 9100,按同一协议把文件名与内容发送出去;测试脚本用 `requests.get("http://127.0.0.1:8081/files")` 验证列表接口。对照检查清单:`sendall` 发送(是)、`recv_exact` 收满(是)、长度上限检查(是)、超时(连接与读取都有)、异常带上下文(是)、HTTP 按 `Content-Length` 读(是)、with 管理资源(是)。

下一章预告:第9章 异常处理与并发——try/except 完整体系、GIL 是什么与为什么、threading 与 queue、asyncio 入门。让程序在错误与并发面前保持优雅。